

Everything in V2 (the v2_8gb set — the current one), organized by what each piece does:

The wire protocol — five TLV types

Type	Code	Carries	Purpose
HIT	0xfd	32-B hash	"You have this chunk — reconstruct it locally"
MISS	0xbf	4-B sender_cid + data	Real chunk data, now

			tagged with the sender's own chunk ID
NAK	0xfb	32-B hash	"You sent a HIT I can't resolve — resend the data"
HELLO	0xfe	4-B generation	Sent once per connection, before any chunk — store-

			incarnati on handshak e
CONFIRM	0xfc	$N \times 4\text{-B}$ cids	Receiver reports which sender_ci ds it durably stored

The five mechanisms

1. Epoch/generation (module2 + module4) — each store has a wall-clock generation persisted in `Chunk_store.gen`, bumped only when the store

file has to be created fresh (a wipe). HELLO advertises it; a peer seeing a changed generation marks that peer cold (MISS-only) and wipes its confirmed bitmap. Handles the full-wipe case with zero NAKs.

2. NAK + resend (module4) — the last-resort safety net: unresolvable HIT → NAK back on so1 → sender resends as MISS. A burst of 5 NAKs flips the peer cold (single NAKs are normal post-eviction traffic).

3. Stall-at-gap reorder queue (module4 + state in module1_state) — the fix for the corruption you caught. An unresolvable HIT enqueues a hole at its stream position; everything after it queues *behind* the hole instead of overtaking; the NAK-resend is matched to the oldest hole by hash, takes its position, and the queue drains in order. Fast path (no hole outstanding) forwards directly

with zero overhead. Capped at 1024 records with a loud counted force-drain.

4. sender_cid + CONFIRM + confirmed-set bitmap (module4) — the partial-divergence answer. Every MISS carries the sender's cid; the receiver batches up to 64 stored cids and CONFIRMs them back; the sender keeps a per-peer bitmap (1 bit per cid, ~1 MB/peer, 16 peers) and the HIT condition is strict: *local hit AND peer warm AND that exact cid confirmed* — otherwise MISS. No wrong HIT ever leaves in steady state, so the stall almost never fires. A NAK clears the offending bit; a generation change clears the whole bitmap.

5. 8 GB slot store + CLOCK eviction (module2) — 2,097,152 fixed 4,136-B slots, file hard-bounded at ~8.68 GB, eviction by second-chance CLOCK with a 262k-access grace window (protects HIT-

in-flight chunks), in-place slot reuse. All hot-path lookups $O(1)$: hash→slot and cid→slot via two `rte_hash` tables, free slots via a stack. Zero linear scans except the one-time startup rebuild.

Per file

`module2.h/.c` — the 8 GB slot store, CLOCK eviction, generation persistence, both hash maps, free stack.

`module4.h/.c` — all five TLV types, the per-peer table (generation + cold + nak_burst + bitmap), reorder queue, CONFIRM batching, both TLV builders (plain + cid-prefixed), full stats counters.

`module1.h/.c` — CDC chunking (unchanged logic) + peer_key, hello_sent, the reorder-queue head/tail and CONFIRM accumulator

living in module1_state, HELLO prepended once per connection, per-chunk hex-dump printf removed.

uipc_socket_v2_patch.txt — two edits: stamp peer_key before module1_add_data, and pass &tp1->m1_state (not .rx) into the now-4-arg decode call.

The rules that come with it

Both NAs must run V2 together — a V1 decoder reads the cid prefix as payload → corruption. Never mix.

Wipe Chunk_store.dat on both ends — slot format can't read the old append format.

~152 MB static BSS (index + free stack + bitmaps).

Not in V2 (known, documented): persistence of peer table/bitmaps across restart, the g_peers lock, eviction-notify, telemetry.

The decisive acceptance test remains the one V1-NAK failed: partial divergence, received file byte-identical, reorder_filled > 0, reorder_OVERFLOW == 0.

Honest scorecard, scored against what's *proven*, not what's written. Weakest first, since those define the real grade.

Parameter	Score	Why
Test coverage	2/10	V2 (stall-at-gap,

CONFIRM,
bitmap,
CLOCK
eviction) has
never run on
hardware.
Only NAK-
alone and
epoch have
real two-box
runs — and
NAK-alone
failed its test
(reorder
corruption).
Nothing has
run at 8 GB
scale or with
eviction
actually

		firing.
Stability under load	3/10	RX mbuf leak (~7.6k units/tick) unresolved – the appliance kills itself under sustained traffic regardless of feature quality. Socket-lock patches unconfirmed on your tree. The 4-concurrent-

		FTP test that exposed everything has never fully passed.
Concurrency safety	4/10	Design is now lock-aware (module1 spinlock, MemPool fixes, take_pending), but g_peers+bitmaps are unlocked across lcores, and

		the reorder queue's safety <i>assumes</i> the socket locks are in. Wet concrete.
Restart behavior	4/10	Generation persists (good, time-based, survives full wipes). Peer table + bitmaps don't → one wasted full-MISS transfer per

		peer after every restart. Mid-run store deletion silently orphans the inode.
Observability	4/10	Good counters exist (print_stats on 2 and 4) but only on-demand to stdout. No telemetry — every field problem currently

		requires a terminal and a camera.
Correctness of design	8/10	Every divergence case has a designed, sim-validated answer: wipe→epoch, partial→bitmap, race→NAK+stall-at-gap. The reorder bug was found, root-caused, and structurally

		fixed. Docked 2: stall-at-gap unproven, and known accepted edges (reorder overflow drops, bitmap cid exhaustion).
Wire efficiency	7/10	35 B vs 4 KB per HIT, your own measurement showed 98.7% chunk reuse post-

		edit. Docked for: 37.5% forced MAX- cuts (CDC clamp unfixed), no sequence- level references, no MISS compression.
Store/ algorithm quality	8/10	$O(1)$ everywhere on hot paths (two rte_hash maps, free stack, CLOCK),

		bounded 8.68 GB file, in-place eviction, grace window for the RTT race. Docked: fflush-per- chunk on stores, random-seek I/O pattern at scale unmeasured.
Memory efficiency	7/10	~152 MB BSS for an 8 GB store index is

		reasonable (~1.9%). Bitmaps 1 MB/peer fine. Docked: fixed slots waste disk on partial chunks; cid space exhausts after ~3 store turnovers.
Scalability (multi-node)	6/10	Hub-and-spoke identity, per-peer state, 16 peers —

matches
your
topology.
Untested
beyond 2
boxes; hub-
side steady
state
(permanent
partial
divergence)
is exactly the
untested
path.

Overall: design [8/10](#), product 3-4/10. The gap between those two numbers *is* your remaining work, and it's concentrated in three items: run the V2 partial-divergence test, kill the leak,

confirm the locks. Those three alone would pull the product score to ~7 without writing a single new feature.